UpCharika Medical AI - VectorStore & RAG Master Guide

From 9 Medical Textbooks to FAISS Vectors, Google Drive Sync & Real-Time Groq LLM Inference

1. System Concept: The Smart Hospital Library

Modern LLMs have extensive general vocabulary but lack real-time memory of proprietary medical textbooks. UpCharika implements **Retrieval-Augmented Generation (RAG)** to guarantee medically accurate answers without hallucinations. Think of this system as an automated hospital library:

```
[9 Heavy Medical Books] (Harrison's, Davidson's, BNF 76, KD Tripathi, Robbins...)
       |
       +--> 1. PyMuPDF (The Fast Scanner)       : Extracts text page-by-page with page metadata.
       +--> 2. LangChain Splitter (Paper Cutter): Slices pages into 77,636 cards (1,000 chars each).
       +--> 3. all-MiniLM-L6-v2 (Meaning Tagger): Labels each card with 384 mathematical coordinates.
       +--> 4. FAISS (The Filing Cabinet)       : Stores coordinates for sub-2 millisecond search.
       +--> 5. Google Drive (The Cloud Vault)   : Hosts the 198 MB database bypassing GitHub limits.
       +--> 6. Groq LLM (The Doctor)            : Reads top retrieved cards and diagnoses the patient.
```

2. The Architecture Components: Who Does What?

Clear distinction between all the key frameworks, libraries, and platforms:

Technology	Role & Type	Everyday Analogy	Exact Job in UpCharika
Hugging Face	Cloud Platform (huggingface.co)	?? The Model Showroom / Store	Hosts pre-trained model weights. Allows us to download all-MiniLM-L6-v2 for free without paying API fees.
sentence-transformers	Python Library (Local CPU Engine)	?? The Engine Under the Hood	Executes the neural network on your laptop CPU. Takes medical sentences and outputs 384 floating-point numbers.
all-MiniLM-L6-v2	Transformer Model (6 Layers, 384-D)	?? The Meaning Translator	Trained on >1 billion sentence pairs by Microsoft. Understands that 'severe headache' and 'migraine' mean the same thing.
LangChain	Orchestration Framework	??? The Master Architect / Manager	Connects all components into a clean pipeline: PDF Loader -> Text Splitter -> Embeddings -> FAISS -> Prompt -> Groq LLM.
langchain-community	Integration Adapter	?? The 3rd-Party Bridge	Provides the official Python bridge to load and save Facebook FAISS vector databases (FAISS.from_documents).
langchain-huggingface	Integration Adapter	?? The Hugging Face Bridge	Provides HuggingFaceEmbeddings wrapper to effortlessly invoke sentence-transformers from within LangChain.
FAISS (faiss-cpu)	Vector Search Engine (Facebook AI)	?? The Ultra-Fast Filing Cabinet	Stores the 77,636 vectors. Calculates cosine similarity in 2ms on standard CPU to find the 3 best medical chunks.
Google Drive & gdown	Cloud Storage & Downloader	?? The Cloud Cargo Truck	Bypasses GitHub's 100 MB file limit by hosting vs.zip (180 MB) online. gdown downloads and extracts it automatically on cloud startup.
Groq Cloud API	Inference Engine (LPU Accelerators)	?? The Consulting Physician	Runs Llama-3.1, Qwen-2.5, and GPT-OSS models at 500+ tokens/sec. Reads retrieved book chunks and generates polite clinical answers.

3. Step-by-Step: How the VectorStore was Created (Offline Phase)

The vector store is created **only once** by running python create_memory_for_llm.py. Here is the complete chronological sequence:

Phase	Operation	Code & Technical Mechanics	Output
1. Raw PDF Ingestion	PyMuPDF (fitz) Page Extraction	Iterates through all 9 PDFs in data/ page-by-page. Extracts text and attaches metadata: {'source': 'Harrison.pdf', 'page': 45}.	List of raw page objects with page numbers
2. Text Chunking	RecursiveCharacter TextSplitter	Slices long pages into 1,000-character pieces with 100-character overlap. The 100-character overlap prevents sentences from being cut in half.	**77,636 text chunks**
3. Semantic Embedding	all-MiniLM-L6-v2 (Hugging Face)	Feeds each chunk through 6 transformer layers on CPU. Converts chunk meaning into 384 floating-point numbers. Math: 77,636 chunks x 384 numbers = 29.8M floats.	29.8 Million float32 coordinates
4. FAISS Indexing	db.save_local() Local Serialization	Saves two files inside vectorstore/db_faiss/: - **index.faiss**: 77,636 x 384 x 4 bytes = **119 MB** - **index.pkl**: Serialized docstore with text & page numbers = **79 MB**.	**index.faiss (119 MB)** **index.pkl (79 MB)**
5. Cloud Packaging	ZIP Compression & Google Drive Sync	Compresses db_faiss into vs.zip (~180 MB). Uploads to Google Drive with public view permission. Sets up automatic cloud bootstrap.	Public Google Drive link for deployment

4. The Cloud Deployment Dilemma: Google Drive Integration

GitHub enforces a strict **100 MB per-file limit**. Because index.faiss is 119 MB, pushing it to GitHub fails immediately. The solution is cloud synchronization via Google Drive:

```
# In medibot.py (Lines 46-56):
VECTOR_URL = "https://drive.google.com/uc?id=1UunmpU4iudJycj-OibngzcJf18nA6OmR"

def load_vectorstore():
    # 1. On local laptop: index.faiss exists -> returns immediately (0 seconds)
    if not os.path.exists("vectorstore/db_faiss/index.faiss"):
        with st.spinner("Preparing AI system... please wait"):
            # 2. On cloud server: downloads vs.zip bypassing virus warning
            gdown.download(VECTOR_URL, "vs.zip", quiet=False)
            # 3. Unzips db_faiss folder into vectorstore/
            with zipfile.ZipFile("vs.zip", 'r') as zip_ref:
                zip_ref.extractall("vectorstore")
            # 4. Deletes temporary zip archive to save RAM & disk
            os.remove("vs.zip")
```

5. Real-Time Execution: What Happens When a Patient Asks a Question?

When a user types a query like *'Severe headache with high fever'* into UpCharika:

```
1. USER TYPES QUERY : 'Severe headache with high fever'
        |
        v
2. EMBED QUESTION   : all-MiniLM-L6-v2 converts the question into 384 numbers in ~15 ms.
        |
        v
3. FAISS VECTOR SCAN: FAISS compares the question's 384 numbers against 77,636 vectors in 2 ms.
                      Pulls top 3 matching chunks from Harrison's and Davidson's via index.pkl.
        |
        v
4. BUILD RAG PROMPT : Combines: System Medical Prompt + Top 3 Book Chunks + User Question.
        |
        v
5. GROQ LPU CALL    : Groq API processes prompt at 500+ tokens/sec using Llama-3.1 / Qwen-2.5.
        |
        v
6. RESPONSE DISPLAY : Renders structured medical triage card with symptoms, remedies & citations.
```

6. Viva & Oral Defense Q&A;: Master Questions

Q1: What is the difference between Hugging Face and Sentence-Transformers?

Hugging Face is the online model hub/store where pre-trained model weights are hosted. Sentence-Transformers is the Python library that executes locally on our CPU to run the neural network layers and output 384-dimensional vectors.

Q2: Why did you choose all-MiniLM-L6-v2 over OpenAI embeddings?

all-MiniLM-L6-v2 is completely open-source, runs 100% locally on standard laptop CPUs without needing an internet connection or GPU, has an ~80 MB footprint, runs in 15ms, and incurs zero subscription or token cost.

Q3: Why is index.faiss 119 MB while index.pkl is 79 MB?

index.faiss stores only the mathematical float32 vectors (77,636 chunks x 384 numbers x 4 bytes = 119 MB). index.pkl stores the Python docstore containing the original English paragraphs, book titles, and page numbers (79 MB).

Q4: Why is Google Drive needed if you already have GitHub?

GitHub enforces a strict 100 MB per-file limit. Because index.faiss is 119 MB, GitHub rejects it. We host vs.zip (180 MB) on Google Drive, and medibot.py uses gdown to download and unpack it automatically on cloud deployment.

Q5: What are langchain-community and langchain-huggingface?

They are modular plugin adapters for LangChain. langchain-community provides the connector to Facebook FAISS, while langchain-huggingface provides the wrapper to invoke sentence-transformers embedding models.